

What is Apache Airflow?

Imagine you run a restaurant. Every morning, you need to follow steps in order:

1. Open the doors.
2. Turn on the lights.
3. Clean the tables.
4. Prep the food.

If step 2 fails, you can't prep food in the dark! **Apache Airflow** is your automatic manager. It ensures steps happen in the exact right order, at the right time, and lets you know if something breaks.

In technical terms: **Airflow is an open-source framework used to create, schedule, and monitor data workflows.**

1. Core Definitions & What Airflow Is (and Is Not)

- **Airflow is NOT a Data Streaming framework:** It does not process live data second-by-second (like Kafka or Flink).
- **Airflow is NOT a Data Processing framework:** It does not do the heavy lifting of processing gigabytes of data itself (like Spark or Hadoop).
- **Airflow IS an Orchestration framework:** It acts as the conductor of an orchestra. It tells other tools *when* to start, *what* to run, and *where* to send output.

2. Linear vs Complex (Branching) Workflows

Workflows can run in two ways:

- **Linear Flow:** Simple step-by-step order: `\text{Task A} \rightarrow \text{Task B} \rightarrow \text{Task C}`
- **Complex / Branching Flow:** Parallel tasks and decision-making steps where execution depends on conditions (e.g., if Task A succeeds, run Task B and Task C at the same time; then merge into Task D).

3. Core Components of Airflow

Airflow works like a team with clear individual roles:

Component	Simple Analogy	Responsibility
Airflow DB (Metadata Database)	The Brain / Ledger	Stores all information about tasks, states, credentials, and execution history.
DAG File Processor	The Reader	Parses your Python code files, converts them into workflows, and saves them into the DB.
Airflow Webserver (UI)	The Dashboard	Visual interface where you inspect, trigger, monitor, and debug your workflows.
Scheduler	The Manager	Monitors tasks and decides

Component	Simple Analogy	Responsibility
		<i>which</i> tasks are ready to run and <i>when</i> .
Executor	The Work Assignor	Takes ready tasks from the Scheduler and assigns them to Workers.
Workers	The Laborers	The actual machines/threads that execute your task code.
Triggerer	The Watchman	Efficiently handles long-waiting tasks (async events) without clogging workers.

4. Key Concepts: DAG, Tasks, Operators, and Sensors

- **DAG (Directed Acyclic Graph):**
 - **Directed:** Flows in one direction only.
 - **Acyclic:** No loops allowed ($\text{Task A} \rightarrow \text{Task B} \rightarrow \text{Task A}$ is illegal).
 - **Graph:** A network of connected nodes (tasks).
- **Task:** A single step inside a DAG (e.g., "Download File").
- **Operator:** The template defining *what* a task does.
 - *PythonOperator:* Runs Python code.
 - *BashOperator:* Runs bash commands.
 - *PostgresOperator:* Executes SQL queries inside PostgreSQL.
- **Sensor:** A special type of operator that waits for an event to happen (e.g., waiting for a file to land in S3 or an API to return data) before letting downstream tasks run.

5. DAG Execution Lifecycle

1. You write a Python file defining your DAG and place it in the `dags/` folder.
2. **DAG File Processor** reads the file and writes the metadata to the **Airflow DB**.
3. **Scheduler** checks the DB and sees it's time to run a task $\rightarrow$ sets task state to SCHEDULED then QUEUED.
4. **Executor** picks up the queued task and hands it to a **Worker**.
5. **Worker** runs the task $\rightarrow$ updates status in the DB (RUNNING $\rightarrow$ SUCCESS or FAILED).
6. **Webserver** reads the DB to display real-time status on your UI dashboard.

6. Scheduling, Intervals, and Templating

When defining a DAG, three main parameters control execution:

- **start_date:** The historical date when the DAG becomes active.
- **schedule (or schedule_interval):** How often it runs (e.g., `@daily`, `0 0 * * *`, `@hourly`).
- **end_date (optional):** When the DAG should stop running automatically.

Logical Date vs Run Date

- **Logical Date (Execution Date):** Represents the *beginning* of the time period being processed (e.g., 2026-08-01).
- **Actual Run Date:** The moment the schedule interval finishes (e.g., a daily DAG for 2026-08-01 actually executes at 2026-08-02 00:00:00).

Jinja Templating

Airflow allows dynamic values using double curly braces `{{ }}`:

```
# Pass dynamic execution date to a script
task = BashOperator(
    task_id="print_date",
    bash_command="echo Processing data for date {{ ds }}", # ds =
YYYY-MM-DD
)
```

Quick Reference Summary

`\text{Python File (DAG Definition)}` $\rightarrow$ `\text{Scheduler}` $\rightarrow$ `\text{Executor}` $\rightarrow$ `\text{Worker}` $\rightarrow$ `\text{DB / Web UI}`
 Would you like to write a simple Python DAG example together step-by-step to test your understanding?